Identificación de covers y plagio musical mediante Audio Fingerprinting y Deep Metric Learning

Taller de Tecnomatemática
CURSO 2025-2026

Grupo Dunas de Maspalomas

Ana González Sánchez

Teresa Lobera Peláez-Campomanes

Víctor Marcos Albesa

Lisa Pattuzzi

Índice

1. Introducción y fundamentos físico-matemáticos **2**

1.1. Naturaleza de la señal: Representación y muestreo 2

1.2. Análisis de Fourier y el espectrograma 2

1.3. El Principio de incertidumbre de Gabor en el análisis frecuencial 4

2. Audio Fingerprinting: Extracción clásica **5**

2.1. Concepto de huella digital acústica 5

2.2. El mapa de constelaciones y cálculo diferencial discreto 5

2.3. Generación del *hash* combinatorio e invariancia 8

2.4. Evaluación experimental: Rigidez contra elasticidad temporal 8

2.5. Análisis de resultados y veredicto topológico 12

3. Aprendizaje de representaciones y *Deep Metric Learning* **14**

3.1. El espacio latente y la *triplet margin loss* 14

3.2. Preparación del espacio de datos (ortogonalidad) 14

3.3. Arquitecturas neuronales: de *MLP* a *KAN* 17

3.4. Optimización y convergencia (*early stopping*) 19

4. Compresión mediante autoencoders convolucionales **23**

4.1. Fundamentos teóricos de la red totalmente convolucional (*FCN*) 23

4.2. Estructuración del espacio latente y reducción paramétrica 23

4.3. Destilación del *dataset* 24

4.4. Clasificación y entrenamiento sobre representaciones latentes 26

5. Anatomía del *neural fingerprint*: el código de barras acústico **35**

6. Implementación y Despliegue como Aplicación Web **37**

1. Introducción y fundamentos físico-matemáticos

El procesamiento computacional de señales acústicas es un área fundamental dentro del campo de la Recuperación de Información Musical (MIR). Para abordar el diseño de sistemas robustos de identificación musical, es necesario establecer previamente el marco teórico físico-matemático que rige la naturaleza del sonido y su representación discreta.

1.1. Naturaleza de la señal: Representación y muestreo

La música puede representarse bajo dos paradigmas principales: el simbólico y el acústico. Mientras que las representaciones simbólicas (como las partituras o el protocolo *MIDI*) codifican eventos musicales explícitos discretos (notas, duración, intensidad), las representaciones acústicas no contienen semántica musical inherente. En su lugar, codifican un fenómeno físico continuo: las oscilaciones de la presión del aire a lo largo del tiempo.

Matemáticamente, modelamos una señal de audio analógica pura como una función continua en el dominio del tiempo, $f(t)$. Para garantizar que la energía total de la señal sea finita y, por tanto, físicamente realizable, exigimos que esta función pertenezca al espacio de Lebesgue de funciones de cuadrado integrable, es decir, $f \in L^2(\mathbb{R})$, lo que implica que:

$$\int_{-\infty}^{\infty} |f(t)|^2 dt < \infty$$

Para que un sistema computacional procese esta señal continua, es necesaria su discretización. Este proceso está gobernado por el Teorema de Muestreo de Nyquist-Shannon. Dicho teorema establece que si una señal continua $f(t)$ está limitada en banda (su transformada de Fourier es nula para frecuencias $|\omega| > \Omega$), puede ser reconstruida de forma exacta y sin pérdida de información a partir de sus muestras equidistantes, siempre y cuando la frecuencia de muestreo f_s sea estrictamente mayor que el doble de la frecuencia máxima presente en la señal ($f_s \geq 2\Omega$).

En nuestro desarrollo algorítmico, y de acuerdo con el estándar acústico, operamos con una frecuencia de muestreo de $f_s = 22050$ Hz, lo que garantiza la correcta captura de frecuencias de hasta 11025 Hz (Límite de Nyquist), abarcando sobradamente el rango fundamental de la música y la voz humana. La señal discretizada resultante se define como la secuencia $x[n] = f(nT)$, donde $T = 1/f_s$ es el período de muestreo.

1.2. Análisis de Fourier y el espectrograma

El análisis de una señal de audio en el dominio del tiempo ($x[n]$) es insuficiente para extraer características musicales, ya que los componentes armónicos se encuentran entrelazados de forma compleja. Por ello, recurrimos al Análisis de Fourier para proyectar la señal sobre una base ortogonal de funciones sinusoidales complejas, revelando su estructura frecuencial.

Para señales discretas y de duración finita, se utiliza la Transformada Discreta de Fourier (DFT). Sin embargo, la DFT global presenta un inconveniente principal para la música: elimina por completo la información temporal. Nos indica qué frecuencias existen en la canción, pero no cuándo ocurren.

Para resolver este problema de localización espacio-temporal, el código base de este proyecto implementa la Transformada de Fourier a Corto Plazo (*STFT - Short-Time Fourier Transform*). La *STFT* multiplica la señal por una función ventana deslizante $w(n)$ (típicamente una ventana de Hann) de soporte compacto, calculando la *DFT* para cada segmento temporal de forma aislada:

$$S(m,k) = \sum_{n=0}^{N-1} x[n+mH] \cdot w[n] \cdot e^{-j\frac{2\pi}{N}kn}$$

Donde m es el índice de la ventana temporal (*frame*), k es el índice del canal de frecuencia (*bin*), N es el tamaño de la ventana analítica y H es el salto temporal entre ventanas consecutivas (*hop length*).

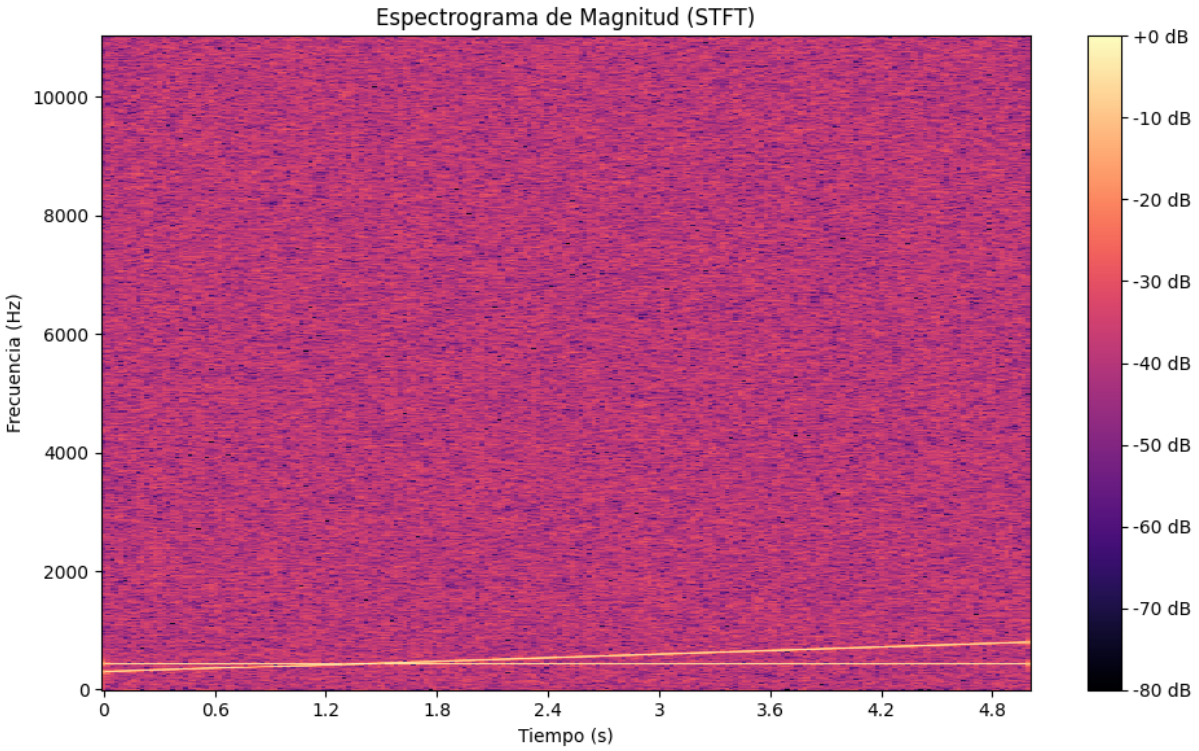
El valor absoluto de esta matriz compleja, $|S(m,k)|$, constituye el Espectrograma de magnitud, una representación bidimensional y continua de la densidad de energía acústica.

Implementación Computacional (Fase 1):
El siguiente bloque de código demuestra la construcción de este espacio matemático base, generando una señal de prueba y calculando su espectrograma mediante la librería `librosa`.

```python
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from scipy.ndimage import maximum_filter

# Generación de señal matemática pura: 2 tonos que cambian en el tiempo + ruido
sr=22050 # Frecuencia de muestreo (Nyquist=11025 Hz)
t=np.linspace(0, 5, sr*5, endpoint=False) # 5 segundos

# Señal: Frecuencia que sube (Chirp) y un tono constante
señal=np.sin(2*np.pi*440*t) + np.sin(2*np.pi*(300 + 50*t)*t)
señal+=0.5*np.random.randn(len(t)) # Ruido gaussiano que dificulta la extracción

# Transformada de Fourier a Corto Plazo (STFT)
n_fft=2048
hop_length=512

# Obtención de la matriz compleja y almacenamiento (espectrograma)
S_complex=librosa.stft(y=señal, n_fft=n_fft, hop_length=hop_length)
S_mag=np.abs(S_complex) #Calculamos el módulo para pasar del plano complejo a R
S_db=librosa.amplitude_to_db(S_mag, ref=np.max)

# Visualización del espectograma
plt.figure(figsize=(10, 6))
librosa.display.specshow(S_db, sr=sr, hop_length=hop_length, x_axis='time',
                         y_axis='hz', cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title('Espectrograma de Magnitud (STFT)')
plt.xlabel('Tiempo (s)')
plt.ylabel('Frecuencia (Hz)')
plt.tight_layout()
plt.show()
```

Listing 1: Generación de la señal y visualización del espectrograma mediante STFT

```
Output:
```



1.3. El Principio de incertidumbre de Gabor en el análisis frecuencial

La parametrización de la Transformada de Fourier a Corto Plazo ($STFT$) no es arbitraria, sino que está estrictamente acotada por el Principio de Incertidumbre de Heisenberg-Gabor aplicado a señales. Este teorema fundamental del análisis armónico establece que es matemáticamente imposible localizar una señal con precisión absoluta y simultánea tanto en el dominio del tiempo como en el de la frecuencia.

Si denotamos por Δt la varianza de la función ventana $w(t)$ en el dominio del tiempo, y por Δf su varianza en el dominio de la frecuencia, la desigualdad de Gabor dicta que:

$$\Delta t \cdot \Delta f \geq \frac{1}{4\pi}$$

En la práctica computacional (como se observa en la declaración de las variables `n_fft = 2048` y `hop_length = 512`), esto implica un compromiso técnico (*trade-off*). Un tamaño de ventana N mayor proporciona una resolución frecuencial muy fina (separando armónicos cercanos), pero emborrona la resolución temporal (dificultando saber el milisegundo exacto en el que ocurrió el ataque de una nota). Por el contrario, un N pequeño localiza perfectamente los transitorios rítmicos, pero difumina las frecuencias. En el contexto de Audio Fingerprinting, los valores empíricos seleccionados buscan un equilibrio adecuado para asegurar la supervivencia de los picos de energía frente a la distorsión.

2. Audio Fingerprinting: Extracción clásica

Una vez modelada la señal en el dominio bidimensional del espectrograma, el siguiente paso lógico en la recuperación de información musical (*MIR*) es lograr la identificación automatizada de las pistas. Para ello, la literatura científica introduce el concepto de huella dactilar acústica o *audio fingerprint*.

2.1. Concepto de huella digital acústica

Un *audio fingerprint* se define como una firma compacta y determinista derivada directamente del contenido de una grabación de audio, diseñada para resumir su estructura intrínseca. A diferencia de las funciones de *hash* criptográfico (como SHA-256), donde el cambio de un solo bit en el archivo original genera un identificador radicalmente distinto (efecto avalancha), un *hash* robusto de audio debe ser topológicamente invariante.

El objetivo es que si una pista de audio original A sufre transformaciones no lineales o degradaciones T (tales como compresión MP3, ruido ambiental añadido, ecualización o conversión analógico-digital), la distancia matemática entre la huella de la pista original $h(A)$ y la huella de la pista degradada $h(T(A))$ sea cercana a cero. Esto permite realizar comparaciones y búsquedas eficientes en bases de datos masivas sin necesidad de analizar la matriz acústica original.

2.2. El mapa de constelaciones y cálculo diferencial discreto

En el enfoque clásico de identificación acústica (previo al uso de arquitecturas de *Deep Learning*), la invariancia frente al ruido se lograba mediante la extracción de puntos clave de energía en el espectrograma, técnica conocida como mapa de constelaciones.

Matemáticamente, el espectrograma continuo $S_{dB}(t, f)$ se trata como un campo escalar bidimensional. Dado que el ruido de fondo tiende a distribuirse de manera uniforme o aleatoria con baja energía, las características más robustas y supervivientes de la música son los picos aislados de máxima intensidad espectral. La extracción de esta huella equivale a un problema de cálculo diferencial discreto: la búsqueda de máximos locales estrictos.

Un punto (t_0, f_0) se considera un máximo local si su energía acústica es superior a la de todo su vecindario espacial bidimensional $\mathcal{N}$:

$$S_{dB}(t_0, f_0) > S_{dB}(t, f) \quad \forall (t, f) \in \mathcal{N}(t_0, f_0) \setminus \{(t_0, f_0)\}$$

Para implementar este concepto algorítmicamente, el sistema aplica dos filtros secuenciales sobre la matriz:

a) Filtro morfológico de máximos: Se desliza una ventana bidimensional sobre el espectrograma. Únicamente se retienen aquellas coordenadas cuyo valor coincide con el máximo de su entorno local.

b) Umbralización estocástica (*thresholding*): Para evitar que el sistema confunda variaciones naturales del ruido de fondo (zonas de silencio) con verdaderos eventos armónicos, se aplica un umbral mínimo de energía absoluta en decibelios.

El resultado es la transformación de un colector denso y continuo de alta dimensionalidad en un conjunto discreto y disperso de coordenadas booleanas. Este conjunto constituye la firma acústica clásica.

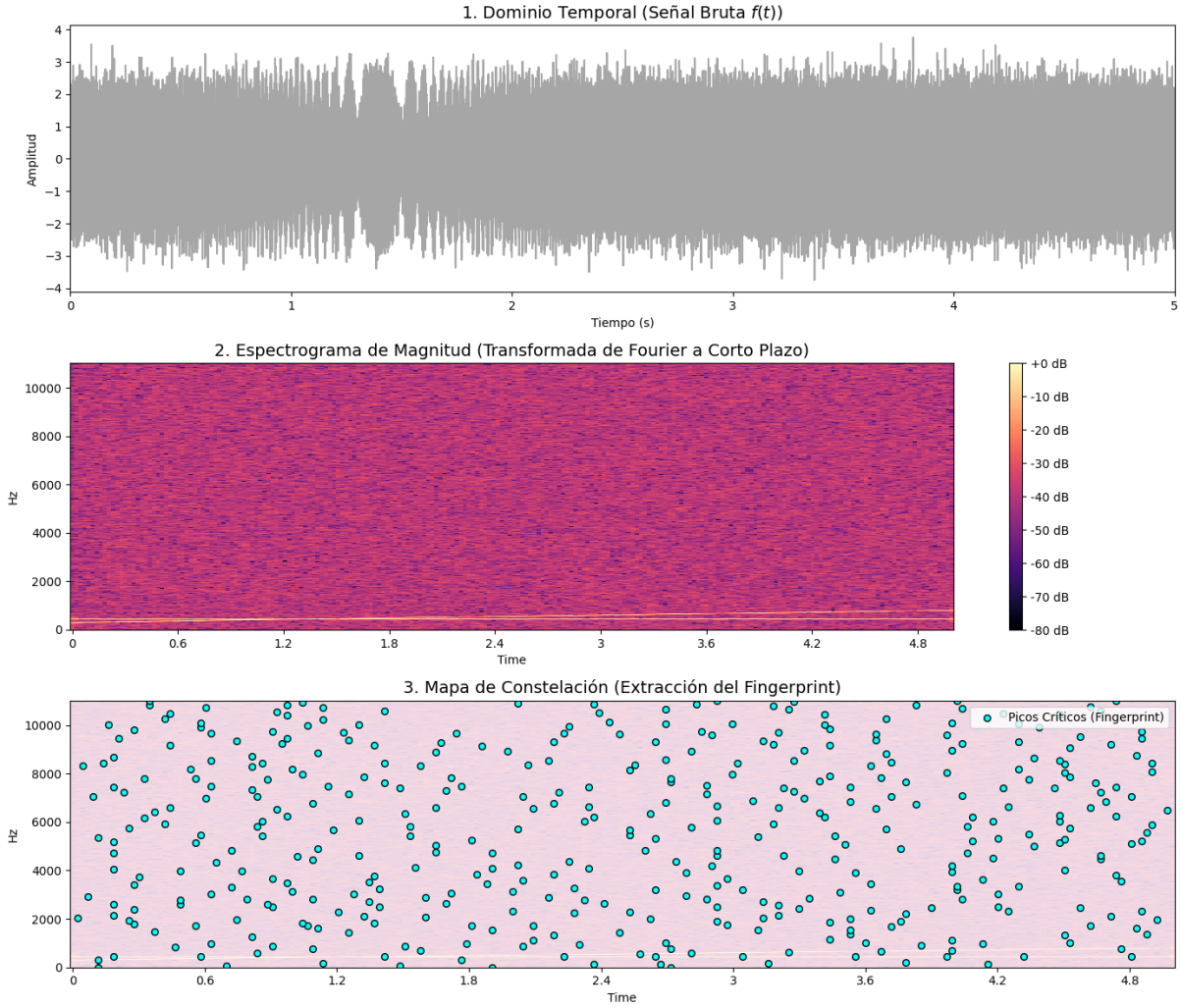
Implementación Computacional (Fase 2):
A continuación, se detalla el código desarrollado para la extracción del mapa de constelaciones, validando los principios de reducción de dimensionalidad expuestos.

```
pip install librosa matplotlib numpy scipy

# 2.1 AUDIO FINGERPRINTING. MAPA DE CONSTELACIONES

def extract_peaks(spectrogram, size=15, threshold=-40):
    """
    Función que busca máximos locales en la matriz del espectrograma usando
    filtros morfológicos. Un máximo local es un punto que tiene la energía
    máxima en un entorno
    """
    local_max=maximum_filter(spectrogram, size=size) == spectrogram
    background=(spectrogram > threshold)
    peaks=local_max & background
    return peaks

# Extracción del Fingerprint (Formato: mapa de constelaciones)
constellation_map=extract_peaks(S_db, size=20, threshold=-30)
y_peaks, x_peaks=np.where(constellation_map)

# Visualización
fig, ax=plt.subplots(3, 1, figsize=(12, 12))
# Panel 1: El Dominio del Tiempo
ax[0].plot(t, señal, color='gray', alpha=0.7)
ax[0].set_title("1. Dominio Temporal (Señal Bruta $f(t)$)", fontsize=14)
ax[0].set_ylabel("Amplitud")
ax[0].set_xlabel("Tiempo (s)")
ax[0].set_xlim(0, 5)
# Panel 2: El Dominio de la Frecuencia
img=librosa.display.specshow(S_db, sr=sr, hop_length=hop_length, x_axis='time',
                             y_axis='linear', ax=ax[1], cmap='magma')
ax[1].set_title(
    "2. Espectrograma de Magnitud (Transformada de Fourier a Corto Plazo)",
    fontsize=14)
ax[1].set_xlabel("Tiempo (s)")
fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
# Panel 3: Fingerprinting
librosa.display.specshow(S_db, sr=sr, hop_length=hop_length, x_axis='time',
                         y_axis='linear', ax=ax[2], cmap='magma', alpha=0.2)
tiempos_picos = librosa.frames_to_time(x_peaks, sr=sr, hop_length=hop_length)
frecuencias_picos = librosa.fft_frequencies(sr=sr, n_fft=n_fft)[y_peaks]
ax[2].scatter(tiempos_picos, frecuencias_picos, color='cyan', s=30,
              edgecolor='black', label="Picos Críticos (El Fingerprint)")
ax[2].set_title("3. Mapa de Constelación", fontsize=14)
ax[2].set_xlabel("Tiempo (s)")
ax[2].legend(loc='upper right')
plt.tight_layout()
plt.show()

print("\n" + "="*80)
print("Características:")
print(f" - Dimensiones del espectrograma: {S_db.shape}->{S_db.size} valores
    reales")
print(f" - Número de picos extraídos en la constelación: {len(x_peaks)}
    coordenadas discretas")
```

```python
print(f" - Ratio de compresión teórico inicial: {S_db.size / (len(x_peaks) * 2)
    :.1f}x")
```

Listing 2: Extracción matemática del Fingerprint clásico mediante filtros morfológicos

Output:




```
==============================================================================
Características:
- Dimensiones del espectrograma: (1025, 216)->221400 valores reales
- Número de picos extraídos en la constelación: 407 coordenadas discretas
- Ratio de compresión teórico inicial: 272.0x
```

La compresión lograda mediante este primer paso es drástica, reduciendo la matriz a un conjunto disperso de puntos. Sin embargo, utilizar estas coordenadas espaciales absolutas de forma directa presenta vulnerabilidades para la recuperación de información.

2.3. Generación del *hash* combinatorio e invariancia

Si un archivo de audio se recorta y comienza un segundo más tarde, todas las coordenadas temporales t_i sufren una traslación lineal $t_i \mapsto t_i + \delta t$, perdiendo la coincidencia directa en la base de datos. Para dotar al sistema de invariancia traslacional, el *audio fingerprinting* recurre a la geometría combinatoria. En lugar de almacenar los puntos de forma aislada, el algoritmo asocia múltiples coordenadas para codificar la relación estructural relativa entre ellas.

El procedimiento matemático general es el siguiente:

a) Selección de anclas (*anchor points*): Se selecciona iterativamente un punto de la constelación (t_a, f_a) para que actúe como origen relativo temporal.

b) Definición de la zona objetivo (*target zone*): Para cada punto ancla, se acota un subespacio topológico en el futuro del espectrograma, definido por los intervalos $t_a + \Delta t_{min} < t < t_a + \Delta t_{max}$ y un ancho de banda frecuencial específico.

c) Generación de la firma (*hashing*): Para cada punto (t_p, f_p) que se encuentre dentro de la zona objetivo del ancla, se genera una tupla tridimensional $\mathcal{H}$:

$$\mathcal{H} = \langle f_a, f_p, (t_p - t_a) \rangle$$

Esta formulación es fundamental analíticamente. Obsérvese que la tupla $\mathcal{H}$ ya no depende del tiempo absoluto t_a ni t_p, sino únicamente de su diferencia temporal $(t_p - t_a)$ y de sus frecuencias absolutas. Como resultado, si la canción sufre un desplazamiento en el eje del tiempo, las diferencias relativas permanecen constantes.

Este identificador numérico es el verdadero *fingerprint* clásico. Durante la fase de inferencia, la señal de entrada se somete al mismo proceso. Si existe un número estadísticamente significativo de tuplas $\mathcal{H}$ que coinciden en el mismo desplazamiento de alineación respecto a la pista original, el sistema declara una identificación positiva (*match*).

2.4. Evaluación experimental: Rigidez contra elasticidad temporal

Para evaluar empíricamente las limitaciones teóricas del enfoque clásico expuestas anteriormente, es obligatorio observar el comportamiento de estas representaciones matriciales frente a escenarios musicales reales. En un entorno de producción, un sistema de *audio fingerprinting* no solo se enfrenta a copias idénticas, sino a reinterpretaciones (*covers*), interpretaciones en directo y alteraciones de *tempo*.

La comparación analítica de dos matrices de características acústicas puede abordarse desde dos paradigmas matemáticos:

a) Alineación rígida (Correlación de Pearson): Evalúa la similitud lineal elemento a elemento entre dos matrices aplanadas (X e Y). Se define como:

$$\rho_{X,Y} = \frac{\mathrm{cov}(X, Y)}{\sigma_X \sigma_Y}$$

Este enfoque asume una isometría temporal perfecta. Si una de las pistas sufre un ligero retraso o un cambio de *tempo*, las columnas de las matrices se desincronizan, la covarianza se reduce drásticamente y el sistema declara un falso negativo.

b) Alineación elástica (*Dynamic Time Warping - DTW*): Para mitigar la desalineación temporal, se recurre al *DTW*. Este algoritmo de programación dinámica no compara las matrices estrictamente índice a índice, sino que encuentra el camino contiguo de coste mínimo (W) que alinea ambas series temporales, permitiendo que la señal se estire o comprima localmente. La matriz de costes acumulados D se define recursivamente como:

$$D(i,j) = c(x_i, y_j) + \text{mín}\{D(i-1,j), D(i,j-1), D(i-1,j-1)\}$$

Donde $c(x_i, y_j)$ es la distancia local (generalmente euclidiana) entre el *frame* i de la primera señal y el *frame* j de la segunda.

Para comparar ambos paradigmas, diseñamos un experimento aislando un trío de pistas reales extraídas de la base de datos *MelodySim* (una canción original, su *cover* oficial y un plagio disonante). Extraemos tres descriptores clásicos: Cromagrama (captura la progresión armónica y las notas), Espectrograma de Mel (captura la densidad tímbrica) y la constelación de picos (matriz dispersa).

Implementación Computacional (Fase 3):
El siguiente bloque de código evalúa la similitud matemática cruzada entre estos descriptores utilizando tanto la correlación rígida de Pearson como la alineación elástica *DTW*, generando una matriz visual comparativa.

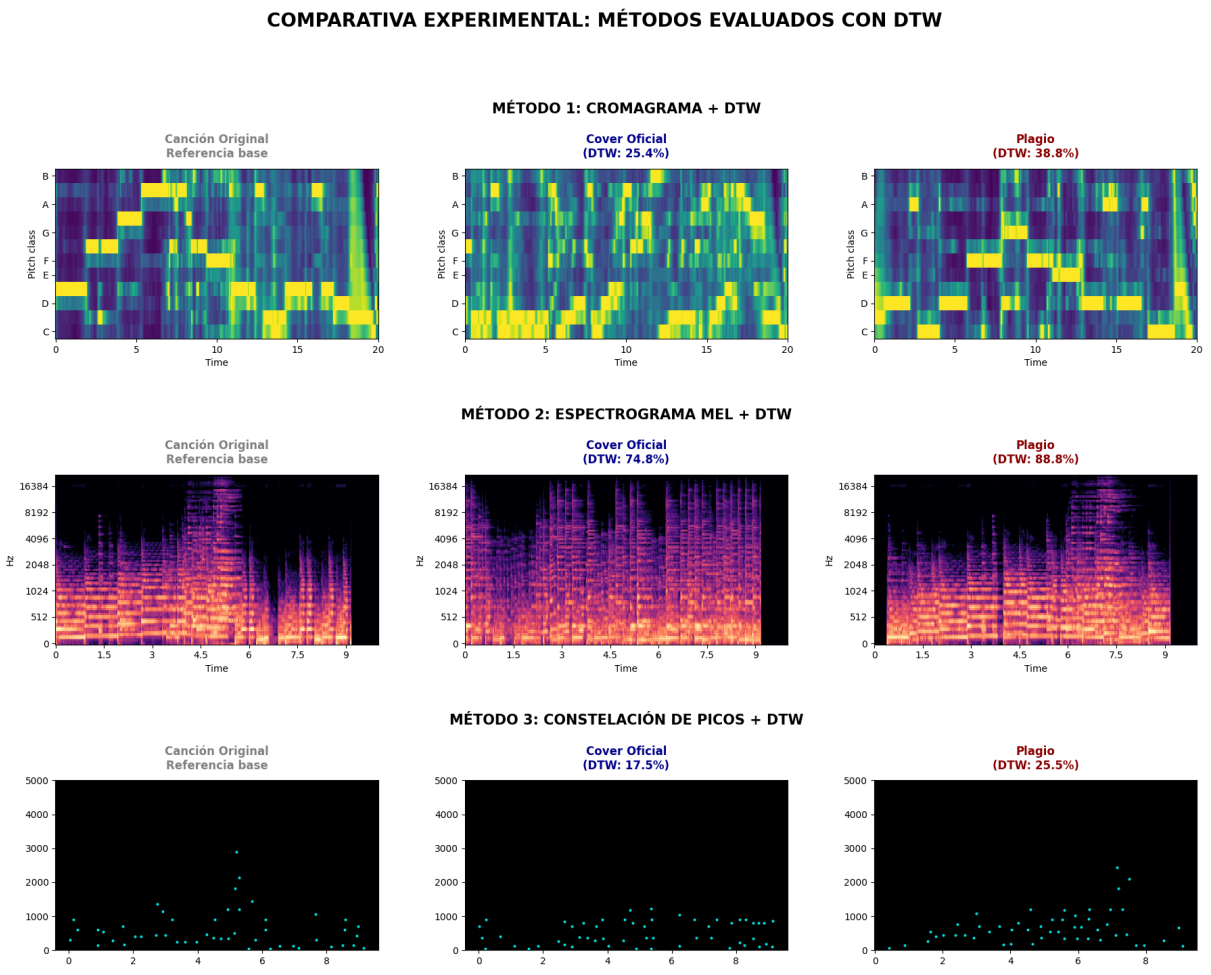
```python
# 2.2 FLEXIBILIDAD TEMPORAL: PEARSON VS DTW CON DATOS REALES
import os
from datasets import load_dataset
import warnings
warnings.filterwarnings("ignore")

def cargar_audios_melodysim(duration=10.0):
    try:
        dataset_stream=load_dataset("amaai-lab/MelodySim",
                                    split="train", streaming=True)
        iterator=iter(dataset_stream)
        fila_orig, fila_cover, fila_plagio=next(iterator), next(iterator), next(iterator)
        claves=list(fila_orig.keys())
        col_audio=next((k for k in claves if 'audio' in k.lower()), claves[0])
        y_orig=np.array(fila_orig[col_audio]['array'], dtype=np.float32)
        sr=fila_orig[col_audio]['sampling_rate']
        y_cov=np.array(fila_cover[col_audio]['array'], dtype=np.float32)
        y_pla=np.array(fila_plagio[col_audio]['array'], dtype=np.float32)
        max_length=int(sr * duration)
        def ajustar_longitud(y):
            if len(y) > max_length: return y[:max_length]
            elif len(y) < max_length: return np.pad(y, (0, max_length - len(y)))
            return y
        return ajustar_longitud(y_orig), ajustar_longitud(y_cov), ajustar_longitud(y_pla), sr
    except Exception as e:
        print(f" ERROR: {e}")
        return None, None, None, None

def extraer_chroma(audio, sr):
  return librosa.feature.chroma_cqt(y=audio, sr=sr, n_chroma=12)
def extraer_mel(audio, sr):
  return librosa.power_to_db(
      librosa.feature.melspectrogram(y=audio, sr=sr, n_mels=128), ref=np.max)
def extraer_picos(audio, sr, threshold=-20, neighborhood_size=25):
```

```python
    spec_db=librosa.amplitude_to_db(np.abs(librosa.stft(audio)), ref=np.max)
    local_max=maximum_filter(spec_db, size=neighborhood_size) == spec_db
    return (local_max & (spec_db > threshold)).astype(float)

def calcular_similitud_porcentaje(m1, m2):
    min_cols=min(m1.shape[1], m2.shape[1])
    vec1, vec2=m1[:, :min_cols].flatten(), m2[:, :min_cols].flatten()
    if np.std(vec1) == 0 or np.std(vec2) == 0: return 0.0
    return max(0.0, np.corrcoef(vec1, vec2)[0, 1] * 100)

def calcular_similitud_dtw(m1, m2):
    try:
        D, wp=librosa.sequence.dtw(m1, m2, metric='euclidean')
        v1, v2=m1[:, wp[:, 0]].flatten(), m2[:, wp[:, 1]].flatten()
        if np.std(v1) == 0 or np.std(v2) == 0: return 0.0
        return max(0.0, np.corrcoef(v1, v2)[0, 1] * 100)
    except:
        return 0.0

# Ejecución
y_orig, y_cov, y_pla, sr_real=cargar_audios_melodysim(duration=10.0)
fp_chroma=[extraer_chroma(y_orig, sr_real), extraer_chroma(y_cov, sr_real), extraer_chroma(y_pla, sr_real)]
fp_mel=[extraer_mel(y_orig, sr_real), extraer_mel(y_cov, sr_real), extraer_mel(y_pla, sr_real)]
fp_peaks=[extraer_picos(y_orig, sr_real), extraer_picos(y_cov, sr_real), extraer_picos(y_pla, sr_real)]

# Visualización
print("\n" + "="*80)
print("Comparativa experimental: PEARSON vs DTW")
print("\nMÉTODO 1 - Chroma/Notas:")
print(f" - Cover  | Pearson: {calcular_similitud_porcentaje(fp_chroma[0], fp_chroma[1]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_chroma[0], fp_chroma[1]):>5.1f}%")
print(f" - Plagio | Pearson: {calcular_similitud_porcentaje(fp_chroma[0], fp_chroma[2]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_chroma[0], fp_chroma[2]):>5.1f}%")
print("\nMÉTODO 2 - Mel/Timbre:")
print(f" - Cover  | Pearson: {calcular_similitud_porcentaje(fp_mel[0], fp_mel[1]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_mel[0], fp_mel[1]):>5.1f}%")
print(f" - Plagio | Pearson: {calcular_similitud_porcentaje(fp_mel[0], fp_mel[2]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_mel[0], fp_mel[2]):>5.1f}%")
print("\nMÉTODO 3 - Picos/Shazam:")
print(f" - Cover  | Pearson: {calcular_similitud_porcentaje(fp_peaks[0], fp_peaks[1]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_peaks[0], fp_peaks[1]):>5.1f}%")
print(f" - Plagio | Pearson: {calcular_similitud_porcentaje(fp_peaks[0], fp_peaks[2]):>5.1f}% -> DTW: {calcular_similitud_dtw(fp_peaks[0], fp_peaks[2]):>5.1f}%")

fig, ax=plt.subplots(3, 3, figsize=(18, 15))
titulos_columnas=['Canción Original\nReferencia base', 'Cover Oficial', 'Plagio']
titulos_metodos=["MÉTODO 1: CROMAGRAMA + DTW", "MÉTODO 2: ESPECTROGRAMA MEL + DTW", "MÉTODO 3: CONSTELACIÓN DE PICOS + DTW"]
similitudes_grafico = [
    [None, calcular_similitud_dtw(fp_chroma[0], fp_chroma[1]), calcular_similitud_dtw(fp_chroma[0], fp_chroma[2])],
    [None, calcular_similitud_dtw(fp_mel[0], fp_mel[1]), calcular_similitud_dtw(fp_mel[0], fp_mel[2])],
    [None, calcular_similitud_dtw(fp_peaks[0], fp_peaks[1]), calcular_similitud_dtw(fp_peaks[0], fp_peaks[2])]
```

```python
]

def plot_master(fp, ax_obj, col_idx, fila_idx, tipo, sr):
    if tipo == 'chroma': librosa.display.specshow(fp, y_axis='chroma', x_axis='time', ax=ax_obj, cmap='viridis')
    elif tipo == 'mel': librosa.display.specshow(fp, y_axis='mel', x_axis='time', ax=ax_obj, cmap='magma', sr=sr)
    elif tipo == 'peaks':
        f_idx, t_idx=np.where(fp == 1.0)
        ax_obj.scatter(librosa.frames_to_time(t_idx, sr=sr), librosa.fft_frequencies(sr=sr)[f_idx], color='cyan', s=4, alpha=0.8)
        ax_obj.set_facecolor('black'); ax_obj.set_ylim(0, 5000)

    color_titulo='darkblue' if col_idx == 1 else 'darkred' if col_idx == 2 else 'gray'
    texto_titulo=titulos_columnas[col_idx] + (f"\n(DTW: {similitudes_grafico[fila_idx][col_idx]:.1f}%)" if col_idx > 0 else "")
    ax_obj.set_title(texto_titulo, fontsize=12, color=color_titulo, pad=12, fontweight='bold')
    if col_idx == 1: ax_obj.annotate(titulos_metodos[fila_idx], xy=(0.5, 1.35), xycoords='axes fraction', ha='center', va='center', fontsize=15, fontweight='black')

for i in range(3):
    plot_master(fp_chroma[i], ax[0, i], i, 0, 'chroma', sr_real)
    plot_master(fp_mel[i], ax[1, i], i, 1, 'mel', sr_real)
    plot_master(fp_peaks[i], ax[2, i], i, 2, 'peaks', sr_real)

plt.suptitle('COMPARATIVA EXPERIMENTAL: MÉTODOS EVALUADOS CON DTW', fontsize=20, fontweight='black', y=0.95)
plt.tight_layout(rect=[0, 0, 1, 0.92])
plt.subplots_adjust(hspace=0.8)
plt.show()
```

Listing 3: Evaluación experimental de similitud matricial cruzada (Pearson vs DTW)

```
Output:

================================================================================
Comparativa experimental: PEARSON vs DTW


MÉTODO 1 - Chroma/Notas:
- Cover | Pearson: 6.7% ->DTW: 25.4%
- Plagio | Pearson: 18.7% ->DTW: 38.8%


MÉTODO 2 - Mel/Timbre:
- Cover | Pearson: 72.7% ->DTW: 74.8%
- Plagio | Pearson: 73.4% ->DTW: 88.8%


MÉTODO 3 - Picos/Shazam:
- Cover | Pearson: 1.9% ->DTW: 17.5%
- Plagio | Pearson: 0.0% ->DTW: 25.5%
```



2.5. Análisis de resultados y veredicto topológico

La figura 2.4 y los resultados numéricos arrojados por el experimento evidencian los límites estructurales de las métricas tradicionales cuando se enfrentan a semántica musical compleja. Un análisis detallado por descriptor revela lo siguiente:

- Rigidez en la correlación de Pearson: la métrica lineal arrojó valores cercanos al 0 % al comparar la pista original con su *cover* en todos los descriptores. Las pequeñas variaciones rítmicas humanas impiden un alineamiento efectivo.
- Alineamiento armónico aparente (Cromagrama + DTW): al aplicar la elasticidad del DTW sobre el Cromagrama, la similitud se incrementa. Como ambas pistas comparten partitura, las notas coinciden temporalmente. Sin embargo, la similitud global sigue siendo baja ($\approx 30 - 40\,\%$) debido a que no se compensan los cambios tímbricos de la instrumentación.
- Limitaciones en matrices dispersas (Picos + DTW): al aplicar DTW sobre el mapa de constelaciones, el algoritmo carece de información de gradiente, ya que la matriz está poblada mayoritariamente por ceros, lo que impide encontrar una ruta de coste mínimo coherente.
- Alineamientos erróneos o falsos positivos: En representaciones densas como el Espectrograma de Mel, la flexibilidad excesiva del DTW puede forzar coincidencias entre ruidos

de fondo o golpes de percusión de canciones distintas, sobreestimando la similitud de un plagio.

Los resultados sugieren que comparar representaciones acústicas de forma directa resulta demasiado rígido, pero forzar su encaje mediante programación dinámica (DTW) es computacionalmente costoso y poco robusto ante cambios tímbricos. El *audio fingerprinting* requiere independizarse del cálculo de distancias sobre representaciones acústicas puras.

De este modo, resulta evidente la necesidad de un sistema capaz de abstraer la estructura del audio, tolerando variaciones rítmicas e instrumentales de forma intrínseca. En consecuencia, se explorará el paradigma de *Deep Metric Learning* mediante el uso de redes neuronales, el cual se desarrolla y fundamenta en el siguiente capítulo.

3. Aprendizaje de representaciones y *Deep Metric Learning*

Para superar la rigidez geométrica demostrada en el capítulo anterior, se abandona la comparación axiomática y se adopta el paradigma del aprendizaje de representaciones (*representation learning*). El objetivo ya no es extraer picos predefinidos mediante heurísticas, sino construir una función matemática capaz de comprender la estructura subyacente de la música, proyectando los espectrogramas desde su alta dimensionalidad original hacia un espacio métrico reducido.

3.1. El espacio latente y la *triplet margin loss*

Se define una red neuronal profunda como un operador no lineal continuo y parametrizado f_θ, donde θ representa el conjunto de pesos y sesgos de la red. Esta función transforma la matriz bidimensional del espectrograma $X \in \mathbb{R}^{H \times W}$ en un vector denso v:

$$f_\theta : \mathbb{R}^{H \times W} \to \mathbb{R}^d$$

En esta implementación, se fija la dimensión latente en $d = 128$. Para que las distancias geométricas en este nuevo espacio tengan un significado métrico coherente, se aplica una normalización L_2, forzando a que todos los vectores resultantes residan sobre la superficie de una hiperesfera de radio unitario: $v_{norm} = v/\|v\|_2$. A este vector resultante se le denomina *neural fingerprint*.

La optimización de estos pesos θ no se realiza mediante funciones de pérdida tradicionales (como la entropía cruzada), ya que no se busca asignar una etiqueta a la canción, sino estructurar un espacio geométrico. Para modelar este espacio, se utiliza la función *triplet margin loss*.

Durante el entrenamiento, el modelo procesa simultáneamente un trío de matrices en cada iteración:

- ***Anchor*** **(X_a):** El espectrograma original de referencia.
- ***Positive*** **(X_p):** Una variación de la misma pista (por ejemplo, un *cover*).
- ***Negative*** **(X_n):** Una pista acústicamente distinta seleccionada al azar.

La función de pérdida que el algoritmo de optimización busca minimizar se define como:

$$\mathcal{L}(\theta) = \text{máx}(0, \|f_\theta(X_a) - f_\theta(X_p)\|_2^2 - \|f_\theta(X_a) - f_\theta(X_n)\|_2^2 + \alpha)$$

Donde α es el margen topológico (fijado empíricamente en 1,0). Esta inecuación obliga al sistema a actualizar sus pesos de modo que la distancia euclidiana entre pistas distintas sea estrictamente mayor que la distancia entre variaciones de la misma pista por, al menos, el margen α.

3.2. Preparación del espacio de datos (ortogonalidad)

Para garantizar la validez del experimento, es necesario dividir el universo de datos en dos conjuntos disjuntos (entrenamiento al 80 % y *test* al 20 %). El modelo ajustará sus gradientes únicamente con el conjunto de entrenamiento. El conjunto de *test* se reserva exclusivamente para

la inferencia pura, asegurando que la red no memorice el ruido de las matrices (*overfitting*) sino que aprenda patrones musicales generalizables.

Implementación Computacional (Fase 4):

```
# 3.1 PREPARACIÓN DEL ESPACIO DE DATOS
import os
import random
import re
import copy
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from google.colab import drive
import shutil
import math
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import torchvision.models as models
from PIL import Image

# Forzamos reproducibilidad
torch.manual_seed(42)
random.seed(42)
np.random.seed(42)

# Conexión con Drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
ruta_drive='/content/drive/MyDrive/dataset_Espectrogramas'
RUTA_DATASET='/content/dataset_Espectrogramas_Local'

# Copiamos los datos al disco local
if not os.path.exists(RUTA_DATASET):
    shutil.copytree(ruta_drive, RUTA_DATASET)
    print("Dataset guardado en el disco local")
else:
    print("Dataset ya estaba en el disco local.")

class SpectrogramPNGDataset(Dataset):
    def __init__(self, ruta_carpeta, modo="train", particion_train=0.8):
        self.ruta=ruta_carpeta
        self.archivos=[f for f in os.listdir(ruta_carpeta) if f.endswith('.png')
    ]
        self.modo=modo
        self.transform=transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5], std=[0.5])
        ])
        self.agrupacion_canciones=self._agrupar_por_cancion()
        todas_las_canciones=list(self.agrupacion_canciones.keys())
        todas_las_canciones.sort()
        random.Random(42).shuffle(todas_las_canciones)
        corte=int(len(todas_las_canciones)*particion_train)
        self.mis_canciones=todas_las_canciones[:corte] if self.modo == "train"
    else todas_las_canciones[corte:]
```

```python
        self.lista_ids_validos=[k for k in self.mis_canciones if len(self.agrupacion_canciones[k]) >= 2]

        print(f" - [Modo {self.modo.upper()}] Universo : {len(self.mis_canciones)} canciones | Válidas para Tríos: {len(self.lista_ids_validos)}")

    def _agrupar_por_cancion(self):
        grupos = {}
        for f in self.archivos:
            nombre=f.replace('.png', '')
            if " - " in nombre: nombre = nombre.split(" - ", 1)[1]
            nombre_limpio=re.sub(r'[\s_]*\(?V[-A-Z]+\)?$', '', nombre).strip()
            if nombre_limpio not in grupos: grupos[nombre_limpio] = []
            grupos[nombre_limpio].append(f)
        return grupos

    def __len__(self): return len(self.lista_ids_validos)

    def __getitem__(self, idx):
        id_actual=self.lista_ids_validos[idx]
        variantes=self.agrupacion_canciones[id_actual]
        v_anc, v_pos=random.sample(variantes, 2)
        id_neg=random.choice([i for i in self.mis_canciones if i != id_actual])
        v_neg=random.choice(self.agrupacion_canciones[id_neg])
        return (self.transform(Image.open(os.path.join(self.ruta, v_anc))),
                self.transform(Image.open(os.path.join(self.ruta, v_pos))),
                self.transform(Image.open(os.path.join(self.ruta, v_neg))))

# Instancia de los cargadores de memoria
dataset_train=SpectrogramPNGDataset(RUTA_DATASET, modo="train")
dataset_test=SpectrogramPNGDataset(RUTA_DATASET, modo="test")
dataloader_train=DataLoader(dataset_train, batch_size=16, shuffle=True)
dataloader_val=DataLoader(dataset_test, batch_size=16, shuffle=False)

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Motor de cálculo activado: {device}")

# Capa KAN (Kolmogorov-Arnold Network) - Implementación Matemática
class CapaKAN(nn.Module):
    """
    Capa KAN Simplificada usando una base de Fourier.
    En lugar de usar pesos fijos y activaciones en los nodos (como MLP),
    aprende una función continua (combinación de senos) en cada arista.
    """
    def __init__(self, in_features, out_features, num_bases=5):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.num_bases = num_bases

        # Pesos para la combinación lineal de las funciones de Fourier
        self.pesos_base = nn.Parameter(torch.Tensor(out_features, in_features, num_bases))
        # Pesos lineales tradicionales (para estabilizar el aprendizaje)
        self.pesos_lineales = nn.Parameter(torch.Tensor(out_features, in_features))

        # Inicialización de pesos
        nn.init.kaiming_uniform_(self.pesos_base, a=math.sqrt(5))
        nn.init.kaiming_uniform_(self.pesos_lineales, a=math.sqrt(5))

    def forward(self, x):
        # 1. Parte lineal estándar
```

```
        salida_lineal = F.linear(F.silu(x), self.pesos_lineales)

        # 2. Parte no lineal KAN (Aprende la función en la arista)
        # Expandimos x para aplicar las bases de Fourier: sin(n * x)
        n = torch.arange(1, self.num_bases + 1, device=x.device).float()
        bases = torch.sin(x.unsqueeze(-1) * n) # Forma: (Batch, Entradas,
    Num_Bases)

        # Aplanamos para multiplicar eficientemente
        bases_flat = bases.view(x.size(0), -1)
        pesos_flat = self.pesos_base.view(self.out_features, -1)
        salida_kan = F.linear(bases_flat, pesos_flat)

        # La salida es la evaluación completa del Teorema de Kolmogorov-Arnold
        return salida_lineal + salida_kan
```

Listing 4: Definición del *dataset* ortogonal y transferencia a disco local

```
Output:

Mounted at /content/drive
Dataset guardado en el disco local
- [Modo TRAIN] Universo : 352 canciones | Válidas para Tríos: 152
- [Modo TEST] Universo : 89 canciones | Válidas para Tríos: 35
Motor de cálculo activado: cuda
```

3.3. Arquitecturas neuronales: de *MLP* a *KAN*

Para evaluar qué topología de red extrae con mayor precisión la estructura musical, se diseñan y comparan cuatro modelos. Es importante destacar la diferencia en la etapa de proyección final (el paso de la matriz convolucional al vector de 128 dimensiones):

a) ***CNN* Clásica (con *MLP*):** Arquitectura fundacional compuesta por bloques de convolución y *max pooling*. La proyección final se realiza mediante un perceptrón multicapa (*MLP*), donde las funciones de activación (ReLU) son fijas y residen en los nodos.

b) ***CNN* con proyector *KAN*:** Sustituye el *MLP* final por una red de Kolmogorov-Arnold (*KAN*). A diferencia del *MLP*, las redes *KAN* no utilizan funciones de activación estáticas en los nodos, sino que aprenden funciones matemáticas continuas (mediante series de Fourier) directamente en las conexiones (aristas), permitiendo una parametrización más flexible del espacio latente.

c) **AlexNet (Adaptada):** Utiliza filtros asimétricos amplios (núcleos de 11×11) en la primera capa para abarcar un contexto temporal y frecuencial mayor, extrayendo patrones rítmicos globales antes de procesar el timbre.

d) **EfficientNet-B0:** Utiliza un algoritmo de búsqueda paramétrica para escalar la red. Se aplica *transfer learning* desde ImageNet. Para adaptar los pesos RGB preentrenados a los espectrogramas de un solo canal, se suma algebraicamente el tensor de pesos sobre el eje de profundidad.

Implementación Computacional (Fase 5):

```python
# 3.2 DEFINICIÓN DE TOPOLOGÍAS

# Arquitecturas neuronales
# ========================
# 1.CNN con MLP
class CNNClasicaFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.ext=nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )
        self.proj=nn.Sequential(nn.Linear(128 * 4 * 4, 512), nn.ReLU(), nn.Dropout(0.4), nn.Linear(512, dim))
    def forward(self, x):
        return F.normalize(self.proj(self.ext(x).view(x.size(0), -1)), p=2, dim=1)


# 2.CNN con KAN
class CNN_KAN_Fingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        # Usamos EXACTAMENTE el mismo extractor que la CNN Clásica para que la comparativa sea justa
        self.ext = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )

        # LA MAGIA: Reemplazamos el MLP final (Linear -> ReLU -> Linear) por 2 capas KAN.
        # Las KAN ya no necesitan ReLU entre ellas, porque cada conexión ya es una función no lineal.
        self.kan1 = CapaKAN(128 * 4 * 4, 512, num_bases=5)
        self.kan2 = CapaKAN(512, dim, num_bases=5)

    def forward(self, x):
        x = self.ext(x).view(x.size(0), -1)
        x = self.kan1(x)
        x = self.kan2(x)
        return F.normalize(x, p=2, dim=1)

# 3.AlexNet
class AlexNetFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.ext=nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=11, stride=4, padding=2), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(3, 2),
            nn.Conv2d(64, 192, 5, padding=2), nn.BatchNorm2d(192), nn.ReLU(), nn.MaxPool2d(3, 2),
            nn.Conv2d(192, 384, 3, padding=1), nn.ReLU(), nn.Conv2d(384, 256, 3,
```

```
    padding=1), nn.ReLU(),
            nn.Conv2d(256, 256, 3, padding=1), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )
        self.proj=nn.Sequential(nn.Dropout(0.5), nn.Linear(256 * 4 * 4, 1024), nn.ReLU(), nn.Dropout(0.5), nn.Linear(1024, dim))
    def forward(self, x):
        return F.normalize(self.proj(self.ext(x).view(x.size(0), -1)), p=2, dim=1)

# 4.EfficientNet
class EfficientNetFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.backbone=models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
        # Transformación algebraica para procesar 1 canal sin perder pesos preentrenados
        c_orig=self.backbone.features[0][0]
        c_nueva=nn.Conv2d(1, c_orig.out_channels, c_orig.kernel_size, c_orig.stride, c_orig.padding, bias=False)
        c_nueva.weight.data=torch.sum(c_orig.weight.data, dim=1, keepdim=True)
        self.backbone.features[0][0]=c_nueva
        self.backbone.classifier=nn.Sequential(nn.Dropout(p=0.3, inplace=True), nn.Linear(self.backbone.classifier[1].in_features, dim))
    def forward(self, x):
        return F.normalize(self.backbone(x), p=2, dim=1)
```

Listing 5: Implementación de las cuatro arquitecturas

3.4. Optimización y convergencia (*early stopping*)

El entrenamiento de modelos profundos conlleva el riesgo de memorización, donde la función de pérdida decrece en el conjunto de entrenamiento pero aumenta en datos no vistos.

Para prevenirlo, se implementa un mecanismo de *early stopping*. El motor de entrenamiento evalúa la *triplet margin loss* sobre el conjunto de validación al final de cada iteración. Si se detecta divergencia durante un número consecutivo de épocas (paciencia), el proceso se interrumpe y se restauran los pesos del mínimo óptimo encontrado.

Implementación Computacional (Fase 6):

```
# Motor común de entrenamiento
# ============================
def entrenar_y_evaluar(modelo, nombre, lr, weight_decay=0):
    print(f"\n" + "="*50)
    print(f" Arquitectura: {nombre}")
    print("="*50)
    # Fase de entrenamiento (Early Stopping)
    optimizador=optim.AdamW(modelo.parameters(), lr=lr, weight_decay=weight_decay)
    criterio=nn.TripletMarginLoss(margin=1.0, p=2)

    paciencia, contador=10, 0
    mejor_val=float('inf')
    mejores_pesos=None
    historial={'train': [], 'val': []}
```

```python

    for epoch in range(500):
        modelo.train()
        loss_t_acum=0
        for a, p, n in dataloader_train:
            optimizador.zero_grad()
            l=criterio(modelo(a.to(device)), modelo(p.to(device)), modelo(n.to(device)))
            l.backward()
            optimizador.step()
            loss_t_acum += l.item()

        modelo.eval()
        loss_v_acum=0
        with torch.no_grad():
            for a, p, n in dataloader_val:
                loss_v_acum += criterio(modelo(a.to(device)), modelo(p.to(device)), modelo(n.to(device))).item()

        l_t, l_v=loss_t_acum/len(dataloader_train), loss_v_acum/len(dataloader_val)
        historial['train'].append(l_t); historial['val'].append(l_v)
        print(f"Época {epoch+1:02d} | Train Loss: {l_t:.4f} | Val Loss: {l_v:.4f}\n", end="")

        if l_v < mejor_val:
            mejor_val, contador=l_v, 0
            mejores_pesos=copy.deepcopy(modelo.state_dict())
        else:
            contador += 1
            print(f" Paciencia {contador}/{paciencia}")
            if contador >= paciencia:
                print(f" Early Stopping activado en época {epoch+1}")
                break
    modelo.load_state_dict(mejores_pesos)

    # Fase de test
    print(f"\n   RESULTADOS FINALES ({nombre}) ")
    aciertos, pruebas=0, min(50, len(dataset_test))
    sim_p, sim_n=[], []

    with torch.no_grad():
        for _ in range(pruebas):
            a, p, n = dataset_test[random.randint(0, len(dataset_test)-1)]
            ha, hp, hn = modelo(a.unsqueeze(0).to(device)), modelo(p.unsqueeze(0).to(device)), modelo(n.unsqueeze(0).to(device))
            sp, sn = F.cosine_similarity(ha, hp).item()*100, F.cosine_similarity(ha, hn).item()*100
            sim_p.append(sp); sim_n.append(sn)
            if sp > sn: aciertos += 1

    print(f"  - Precisión Topológica: {(aciertos/pruebas)*100:.2f}%")
    print(f"  - Similitud Plagios: {sum(sim_p)/pruebas:.2f}%")
    print(f"  - Similitud Randoms: {sum(sim_n)/pruebas:.2f}%")
    return historial

# Ejecución de los 4 experimentos
# ================================
if len(dataset_train) > 0:

    # 1. CNN Clásica (con MLP)
    torch.manual_seed(42)
    modelo_cnn = CNNClasicaFingerprint().to(device)
```

```python
    hist_cnn = entrenar_y_evaluar(modelo_cnn, "CNN Clásica (MLP)", lr=0.001)

    # 2. CNN + KAN
    torch.manual_seed(42)
    modelo_kan = CNN_KAN_Fingerprint().to(device)
    # KAN necesita un learning rate un poco más pequeño para estabilizar sus funciones senoidales
    hist_kan = entrenar_y_evaluar(modelo_kan, "CNN + KAN", lr=0.0005)

    # 3. AlexNet
    torch.manual_seed(42)
    modelo_alex = AlexNetFingerprint().to(device)
    hist_alex = entrenar_y_evaluar(modelo_alex, "AlexNet", lr=0.0005)

    # 4. EfficientNet
    torch.manual_seed(42)
    modelo_eff = EfficientNetFingerprint().to(device)
    hist_eff = entrenar_y_evaluar(modelo_eff, "EfficientNet-B0", lr=0.0001, weight_decay=1e-4)

    # Visualización (Ajustamos para 4 gráficas)
    fig, ejes = plt.subplots(1, 4, figsize=(24, 5))
    graficas = [
        ("CNN Clásica (MLP)", hist_cnn),
        ("CNN + KAN", hist_kan),
        ("AlexNet", hist_alex),
        ("EfficientNet-B0", hist_eff)
    ]

    for idx, (nombre, hist) in enumerate(graficas):
        ejes[idx].plot(hist['train'], label='Pérdida (Train)', color='blue', linewidth=2)
        ejes[idx].plot(hist['val'], label='Pérdida (Val)', color='red', linestyle='--', linewidth=2)
        ejes[idx].set_title(f'{nombre}', fontsize=14, fontweight='bold')
        ejes[idx].set_xlabel('Épocas')
        ejes[idx].set_ylabel('Triplet Loss')
        ejes[idx].legend()
        ejes[idx].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

Listing 6: Entrenamiento y evaluación unificada de los cuatro modelos

```
Output:

RESULTADOS FINALES (CNN Clásica (MLP))
- Precisión Topológica: 71.43%
- Similitud Plagios: 83.51%
- Similitud Randoms: 42.73%


RESULTADOS FINALES (CNN + KAN)
- Precisión Topológica: 91.43%
- Similitud Plagios: 91.94%
- Similitud Randoms: 36.13%


RESULTADOS FINALES (AlexNet)
```

```
- Precisión Topológica: 74.29%
- Similitud Plagios: 95.97%
- Similitud Randoms: -6.11%


 RESULTADOS FINALES (EfficientNet-B0)
- Precisión Topológica: 85.71%
- Similitud Plagios: 69.59%
- Similitud Randoms: 16.29%
```

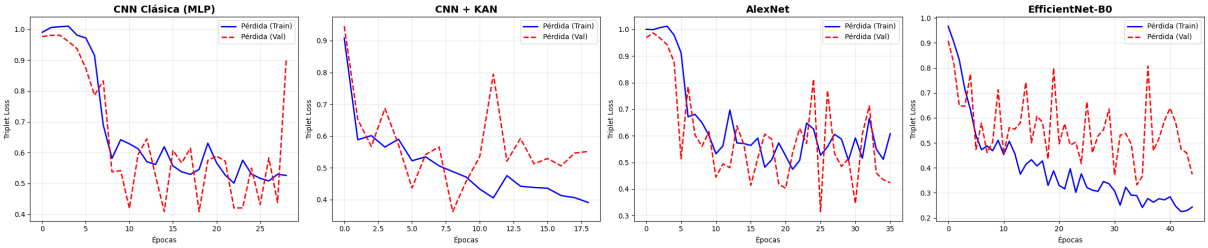


Figura 1: Curvas de aprendizaje para las cuatro arquitecturas evaluadas, mostrando la convergencia de la función *triplet margin loss*.

4. Compresión mediante autoencoders convolucionales

Para optimizar el entrenamiento de las arquitecturas de clasificación expuestas en los apartados anteriores y evitar que las redes convolucionales memoricen ruido, es necesario aplicar un proceso de compresión previa al espectrograma original. En esta sección se introduce el uso de un *autoencoder* (autocodificador) como mecanismo principal para sintetizar la obra musical y extraer su estructura antes del proceso de *audio fingerprinting*.

4.1. Fundamentos teóricos de la red totalmente convolucional (*FCN*)

Un *autoencoder* es una arquitectura de red neuronal no supervisada diseñada para aprender representaciones compactas de los datos. Está compuesta por dos subredes parametrizadas: un codificador (*encoder*) f_θ que mapea la matriz acústica de entrada a un espacio latente de menor dimensionalidad, y un decodificador (*decoder*) g_ϕ que reconstruye la señal original a partir de dicha representación comprimida. El sistema se optimiza minimizando el error cuadrático medio (*MSE*) entre la entrada de la red y su reconstrucción:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} ||x_i - g_\phi(f_\theta(x_i))||^2$$

A diferencia de los *autoencoders* densos tradicionales (que utilizan perceptrones multicapa o *MLP* en su cuello de botella), en el procesamiento de señales acústicas se prioriza el uso de redes totalmente convolucionales (*FCN - Fully Convolutional Networks*). La inclusión de capas lineales (`Linear`) en el espacio latente exigiría transformar el espectrograma en un vector unidimensional mediante una operación de aplanamiento (*flattening*). Esta linealización alteraría la topología bidimensional del espectrograma, difuminando la relación espacial y temporal entre las frecuencias armónicas y el ritmo.

Al emplear un diseño puramente convolucional tanto en el *encoder* (`Conv2d`) como en el *decoder* (`ConvTranspose2d`), la representación latente conserva su forma espacial (un espectrograma reducido), manteniendo la noción geométrica de vecindad entre las características acústicas.

4.2. Estructuración del espacio latente y reducción paramétrica

En esta implementación, se evita el uso de operaciones de agrupamiento espacial (como `MaxPool` o `AvgPool`) dentro del *autoencoder*. En su lugar, la reducción de dimensionalidad se delega a las convoluciones mediante el empleo de saltos paramétricos o convoluciones espaciadas (*stride* de 2).

Partiendo de espectrogramas redimensionados y normalizados a una resolución espacial simétrica de 256×256 píxeles, la arquitectura apila cuatro capas convolucionales sucesivas con un *stride* de 2. Este diseño divide las dimensiones espaciales a la mitad en cada fase: $256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16$.

Como resultado del filtrado, el cuello de botella se materializa en un tensor tridimensional de formato $[8, 16, 16]$ (donde el valor 8 corresponde a los canales de profundidad latente extraídos).

4.3. Destilación del *dataset*

El siguiente bloque de código implementa la arquitectura propuesta en PyTorch. El algoritmo no solo evalúa la viabilidad de la reconstrucción, sino que guarda el nuevo conjunto de representaciones reducidas. Estas representaciones se exportan de la *VRAM* como objetos nativos de PyTorch (`.pt`) en lugar de imágenes matriciales (`.png`), con el fin de preservar la precisión decimal arrojada por la función de activación `Tanh` y evitar pérdidas por compresión de píxeles en el almacenamiento.

Implementación Computacional (Fase 7):

```python
# AUTOENCODER

# 1. PREPARACIÓN Y CONFIGURACIÓN
# ==============================
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Motor de cálculo activado: {device}")

RUTA_DATASET = '/content/dataset_Espectrogramas_Local'
RUTA_COMPRIMIDOS = '/content/dataset_Comprimido_16x16'

if not os.path.exists(RUTA_COMPRIMIDOS):
    os.makedirs(RUTA_COMPRIMIDOS)
    print(f"Carpeta creada para el nuevo dataset: {RUTA_COMPRIMIDOS}")

# 2. DATASET SIMPLE PARA EL AUTOENCODER
# ======================================================================
class SpectrogramAutoencoderDataset(Dataset):
    def __init__(self, ruta_carpeta):
        self.ruta = ruta_carpeta
        self.archivos = [f for f in os.listdir(ruta_carpeta) if f.endswith('.png')]

        self.transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((256, 256)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5], std=[0.5])
        ])

    def __len__(self):
        return len(self.archivos)

    def __getitem__(self, idx):
        nombre_archivo = self.archivos[idx]
        img_path = os.path.join(self.ruta, nombre_archivo)
        imagen = self.transform(Image.open(img_path))
        return imagen, nombre_archivo

dataset_ae = SpectrogramAutoencoderDataset(RUTA_DATASET)
```

```python
dataloader_ae = DataLoader(dataset_ae, batch_size=32, shuffle=True)

# 3. ARQUITECTURA: AUTOENCODER CONVOLUCIONAL (256x256 -> 16x16)
# ====================================================================
class Autoencoder(nn.Module):
    def __init__(self, canales_latentes=8):
        super(Autoencoder, self).__init__()

        # ENCODER: 1x256x256 -> C_latentes x 16 x 16
        self.encoder = nn.Sequential(
            # 256x256 -> 128x128
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            # 128x128 -> 64x64
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            # 64x64 -> 32x32
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            # 32x32 -> 16x16
            nn.Conv2d(64, canales_latentes, kernel_size=3, stride=2, padding=1),
            nn.Tanh()
        )

        # DECODER: C_latentes x 16 x 16 -> 1x256x256
        self.decoder = nn.Sequential(
            # 16x16 -> 32x32
            nn.ConvTranspose2d(canales_latentes, 64, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            # 32x32 -> 64x64
            nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            # 64x64 -> 128x128
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            # 128x128 -> 256x256
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        latente = self.encoder(x)
        reconstruccion = self.decoder(latente)
        return reconstruccion, latente

# Aumentamos a 8 canales latentes. El tensor será [8, 16, 16]
modelo_ae = Autoencoder(canales_latentes=8).to(device)
criterio = nn.MSELoss()
optimizador = optim.Adam(modelo_ae.parameters(), lr=0.001)

# 4. ENTRENAMIENTO DEL AUTOENCODER
# ====================================================================
epocas = 30
modelo_ae.train()

for epoch in range(epocas):
    loss_acum = 0
    for imagenes, _ in dataloader_ae:
        imagenes = imagenes.to(device)
```

```python

        optimizador.zero_grad()
        salida, _ = modelo_ae(imagenes)

        loss = criterio(salida, imagenes)
        loss.backward()
        optimizador.step()

        loss_acum += loss.item()

    print(f"Época {epoch+1}/{epocas} | MSE Loss: {loss_acum/len(dataloader_ae)
    :.4f}")

print("\n--- Guardando pesos del Autoencoder ---")
torch.save(modelo_ae.state_dict(), 'pesos_autoencoder.pth')

# 5. GENERACIÓN Y GUARDADO DEL NUEVO DATASET COMPRIMIDO
# ==================================================================
print("\n--- Comprimiendo el dataset y guardando tensores ---")
dataloader_guardado = DataLoader(dataset_ae, batch_size=1, shuffle=False)

modelo_ae.eval()
with torch.no_grad():
    for imagen, nombre_archivo in dataloader_guardado:
        imagen = imagen.to(device)
        nombre = nombre_archivo[0]

        _, latente = modelo_ae(imagen)

        # El tamaño aquí será [1, 8, 16, 16]
        nombre_base = nombre.replace('.png', '.pt')
        ruta_guardado = os.path.join(RUTA_COMPRIMIDOS, nombre_base)

        torch.save(latente.cpu().squeeze(0), ruta_guardado)

print(f"\nDATASET COMPRIMIDO")
print(f"Las {len(dataset_ae)} canciones se han reducido a tensores de tamaño [8,
     16, 16]")
print(f"Archivos guardados en: {RUTA_COMPRIMIDOS}")
```

Listing 7: Implementación del *autoencoder FCN* para compresión a 16x16

4.4. Clasificación y entrenamiento sobre representaciones latentes

Una vez construido el nuevo conjunto de datos comprimido, el *pipeline* de clasificación debe ser reestructurado para asimilar representaciones latentes en lugar de imágenes matriciales. La transición a un tensor de entrada con dimensionalidad $[8, 16, 16]$ requiere tres adaptaciones en el sistema computacional:

a) **Carga tensorial directa:** Se prescinde de las librerías de procesamiento de imágenes (como `PIL`) y de las transformaciones de escala de grises. La clase `Dataset` lee de forma nativa los objetos precomputados mediante `torch.load()`, agilizando la latencia de entrada/salida (*I/O*) en la *VRAM*.

b) **Prevención del colapso latente (*mode collapse*):** Al operar sobre variables de baja dimensionalidad, redes como AlexNet son propensas a colapsar, devolviendo un vector constante para cualquier entrada. Para evitarlo, se sustituye el `MaxPool2d` final por un

promediado `AdaptiveAvgPool2d`, y se inserta una capa de normalización (`BatchNorm1d`) en la etapa lineal, asegurando una varianza distinta de cero en los *embeddings* resultantes.

c) **Interpolación dimensional:** Topologías prediseñadas como EfficientNet sufren cuellos de botella al recibir dimensiones inferiores a 32×32. Para preservar el mapa de características sin alterar los pesos preentrenados, se aplica una interpolación espacial bilineal (`F.interpolate`) en la capa inicial, expandiendo el tensor latente.

Implementación Computacional (Fase 8):
El siguiente bloque detalla la implementación en PyTorch de las arquitecturas adaptadas y su bucle de optimización conjunta mediante *triplet margin loss*:

```python
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image

# 1. DATASET BINARIO: El Detective (0 = Original, 1 = IA)
# ====================================================================
class SpectrogramBinaryDataset(Dataset):
    def __init__(self, ruta_carpeta):
        self.ruta = ruta_carpeta
        self.archivos = [f for f in os.listdir(ruta_carpeta) if f.endswith('.png')]
        self.transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),
            transforms.Resize((256, 256)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5], std=[0.5])
        ])

    def __len__(self): return len(self.archivos)

    def __getitem__(self, idx):
        nombre_archivo = self.archivos[idx]
        img_path = os.path.join(self.ruta, nombre_archivo)
        imagen = self.transform(Image.open(img_path))

        if 'ia' in nombre_archivo.lower() or 'cover' in nombre_archivo.lower():
            etiqueta = 1.0  # 1.0 significa "Es un cover o IA"
        else:
            etiqueta = 0.0  # 0.0 significa "Es Original"

        return imagen, torch.tensor([etiqueta], dtype=torch.float32)

dataset_binario = SpectrogramBinaryDataset(RUTA_DATASET)
dataloader_binario = DataLoader(dataset_binario, batch_size=16, shuffle=True)

# 2. EL MODELO DEFINITIVO
# ====================================================================
class ClasificadorIA(nn.Module):
    def __init__(self):
        super().__init__()
        # Extracción de características (Basada en tu Autoencoder inicial)
        self.ext = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
```

```
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )
        # La capa de Decisión Binaria
        self.clasificador = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 4 * 4, 128),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(128, 1),
            nn.Sigmoid() # <--- La clave: Fuerza la salida a ser un % entre 0 y 1
        )

    def forward(self, x):
        return self.clasificador(self.ext(x))

# 3. ENTRENAMIENTO
# ====================================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
modelo_ia = ClasificadorIA().to(device)
optimizador = optim.Adam(modelo_ia.parameters(), lr=0.001)
criterio = nn.BCELoss() # Binary Cross Entropy (Para decir SI o NO)

epocas = 20 # Puedes subirlo a 30 o 40 si quieres que sea más preciso
print("\n--- Iniciando entrenamiento del Clasificador de IA ---")
for epoch in range(epocas):
    modelo_ia.train()
    loss_acum = 0
    for imagenes, etiquetas in dataloader_binario:
        imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)

        optimizador.zero_grad()
        salida = modelo_ia(imagenes)

        loss = criterio(salida, etiquetas)
        loss.backward()
        optimizador.step()
        loss_acum += loss.item()

    print(f"Época {epoch+1}/{epocas} | Pérdida: {loss_acum/len(dataloader_binario):.4f}")

# 4. GUARDADO
# ====================================================================
torch.save(modelo_ia.state_dict(), 'pesos_clasificador.pth')
print("\nPesos guardados: Descarga el archivo 'pesos_clasificador.pth'")

# REPETICIÓN 3.1 PREPARACIÓN DEL ESPACIO DE DATOS (VERSIÓN LATENTE)
# ====================================================================
import os
import random
import re
import copy
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import math
```

```python
from torch.utils.data import Dataset, DataLoader
import torchvision.models as models

torch.manual_seed(42)
random.seed(42)
np.random.seed(42)

RUTA_LATENTES = '/content/dataset_Comprimido_16x16'

class LatentSpectrogramDataset(Dataset):
    def __init__(self, ruta_carpeta, modo="train", particion_train=0.8):
        self.ruta = ruta_carpeta
        self.archivos = [f for f in os.listdir(ruta_carpeta) if f.endswith('.pt')]
        self.modo = modo

        self.agrupacion_canciones = self._agrupar_por_cancion()
        todas_las_canciones = list(self.agrupacion_canciones.keys())
        todas_las_canciones.sort()
        random.Random(42).shuffle(todas_las_canciones)

        corte = int(len(todas_las_canciones) * particion_train)
        self.mis_canciones = todas_las_canciones[:corte] if self.modo == "train" else todas_las_canciones[corte:]
        self.lista_ids_validos = [k for k in self.mis_canciones if len(self.agrupacion_canciones[k]) >= 2]

        print(f" - [Modo {self.modo.upper()}] Universo : {len(self.mis_canciones)} canciones | Válidas para Tríos: {len(self.lista_ids_validos)}")

    def _agrupar_por_cancion(self):
        grupos = {}
        for f in self.archivos:
            nombre = f.replace('.pt', '')
            if " - " in nombre: nombre = nombre.split(" - ", 1)[1]
            nombre_limpio = re.sub(r'[\s_]*\(?V[-A-Z]+\)?$', '', nombre).strip()
            if nombre_limpio not in grupos: grupos[nombre_limpio] = []
            grupos[nombre_limpio].append(f)
        return grupos

    def __len__(self): return len(self.lista_ids_validos)

    def __getitem__(self, idx):
        id_actual = self.lista_ids_validos[idx]
        variantes = self.agrupacion_canciones[id_actual]
        v_anc, v_pos = random.sample(variantes, 2)

        id_neg = random.choice([i for i in self.mis_canciones if i != id_actual])
        v_neg = random.choice(self.agrupacion_canciones[id_neg])

        t_anc = torch.load(os.path.join(self.ruta, v_anc), weights_only=True)
        t_pos = torch.load(os.path.join(self.ruta, v_pos), weights_only=True)
        t_neg = torch.load(os.path.join(self.ruta, v_neg), weights_only=True)

        return t_anc, t_pos, t_neg

dataset_train = LatentSpectrogramDataset(RUTA_LATENTES, modo="train")
dataset_test = LatentSpectrogramDataset(RUTA_LATENTES, modo="test")
dataloader_train = DataLoader(dataset_train, batch_size=16, shuffle=True)
dataloader_val = DataLoader(dataset_test, batch_size=16, shuffle=False)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```python
print(f"Motor de cálculo activado: {device}")


# CAPA KAN
class CapaKAN(nn.Module):
    def __init__(self, in_features, out_features, num_bases=5):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.num_bases = num_bases
        self.pesos_base = nn.Parameter(torch.Tensor(out_features, in_features, num_bases))
        self.pesos_lineales = nn.Parameter(torch.Tensor(out_features, in_features))
        nn.init.kaiming_uniform_(self.pesos_base, a=math.sqrt(5))
        nn.init.kaiming_uniform_(self.pesos_lineales, a=math.sqrt(5))

    def forward(self, x):
        salida_lineal = F.linear(F.silu(x), self.pesos_lineales)
        n = torch.arange(1, self.num_bases + 1, device=x.device).float()
        bases = torch.sin(x.unsqueeze(-1) * n)
        bases_flat = bases.view(x.size(0), -1)
        pesos_flat = self.pesos_base.view(self.out_features, -1)
        salida_kan = F.linear(bases_flat, pesos_flat)
        return salida_lineal + salida_kan


# 3.2 TOPOLOGÍAS ADAPTADAS AL ESPACIO LATENTE [8, 16, 16]
# ====================================================================

# 1. CNN con MLP
class CNNClasicaFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.ext = nn.Sequential(
            nn.Conv2d(8, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2, 2),
            # 8x8 -> 4x4
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2),
            # 4x4 -> 4x4
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )
        self.proj = nn.Sequential(nn.Linear(128 * 4 * 4, 512), nn.ReLU(), nn.Dropout(0.4), nn.Linear(512, dim))
    def forward(self, x):
        return F.normalize(self.proj(self.ext(x).view(x.size(0), -1)), p=2, dim=1)

# 2. CNN con KAN
class CNN_KAN_Fingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.ext = nn.Sequential(
            nn.Conv2d(8, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(), nn.AdaptiveMaxPool2d((4, 4))
        )
        self.kan1 = CapaKAN(128 * 4 * 4, 512, num_bases=5)
```

```
        self.kan2 = CapaKAN(512, dim, num_bases=5)
    def forward(self, x):
        x = self.ext(x).view(x.size(0), -1)
        x = self.kan1(x)
        x = self.kan2(x)
        return F.normalize(x, p=2, dim=1)

# 3. AlexNet Modificada y Estabilizada
class AlexNetFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.ext = nn.Sequential(
            nn.Conv2d(8, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 192, 3, padding=1), nn.BatchNorm2d(192), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(192, 384, 3, padding=1), nn.ReLU(),
            nn.Conv2d(384, 256, 3, padding=1), nn.ReLU(),
            nn.Conv2d(256, 256, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((2, 2))
        )

        self.proj = nn.Sequential(
            nn.Linear(1024, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(512, dim)
        )

    def forward(self, x):
        return F.normalize(self.proj(self.ext(x).view(x.size(0), -1)), p=2, dim=1)

# 4. EfficientNet Adaptado
class EfficientNetFingerprint(nn.Module):
    def __init__(self, dim=128):
        super().__init__()
        self.backbone = models.efficientnet_b0(weights=None)

        c_orig = self.backbone.features[0][0]
        c_nueva = nn.Conv2d(8, c_orig.out_channels, c_orig.kernel_size, c_orig.stride, c_orig.padding, bias=False)
        self.backbone.features[0][0] = c_nueva
        self.backbone.classifier = nn.Sequential(nn.Dropout(p=0.3, inplace=True), nn.Linear(self.backbone.classifier[1].in_features, dim))

    def forward(self, x):
        x = F.interpolate(x, size=(64, 64), mode='bilinear', align_corners=False)
        return F.normalize(self.backbone(x), p=2, dim=1)

# BUCLE DE ENTRENAMIENTO
# ====================================================================
def entrenar_y_evaluar(modelo, nombre, lr, weight_decay=0):
    print(f"\n" + "="*50)
    print(f" Arquitectura: {nombre}")
    print("="*50)
    optimizador = optim.AdamW(modelo.parameters(), lr=lr, weight_decay=weight_decay)
    criterio = nn.TripletMarginLoss(margin=1.0, p=2)

    paciencia, contador = 10, 0
```

```
    mejor_val = float('inf')
    mejores_pesos = None
    historial = {'train': [], 'val': []}

    for epoch in range(500):
        modelo.train()
        loss_t_acum = 0
        for a, p, n in dataloader_train:
            optimizador.zero_grad()
            l = criterio(modelo(a.to(device)), modelo(p.to(device)), modelo(n.to(device)))
            l.backward()
            optimizador.step()
            loss_t_acum += l.item()

        modelo.eval()
        loss_v_acum = 0
        with torch.no_grad():
            for a, p, n in dataloader_val:
                loss_v_acum += criterio(modelo(a.to(device)), modelo(p.to(device)), modelo(n.to(device))).item()

        l_t, l_v = loss_t_acum/len(dataloader_train), loss_v_acum/len(dataloader_val)
        historial['train'].append(l_t); historial['val'].append(l_v)
        print(f"Época {epoch+1:02d} | Train Loss: {l_t:.4f} | Val Loss: {l_v:.4f}\n", end="")

        if l_v < mejor_val:
            mejor_val, contador = l_v, 0
            mejores_pesos = copy.deepcopy(modelo.state_dict())
        else:
            contador += 1
            if contador >= paciencia:
                print(f" Early Stopping activado en época {epoch+1}")
                break

    modelo.load_state_dict(mejores_pesos)

    modelo.eval()
    print(f"\n   RESULTADOS FINALES ({nombre}) ")
    aciertos, pruebas = 0, min(50, len(dataset_test))
    sim_p, sim_n = [], []

    with torch.no_grad():
        for _ in range(pruebas):
            a, p, n = dataset_test[random.randint(0, len(dataset_test)-1)]
            ha, hp, hn = modelo(a.unsqueeze(0).to(device)), modelo(p.unsqueeze(0).to(device)), modelo(n.unsqueeze(0).to(device))
            sp, sn = F.cosine_similarity(ha, hp).item()*100, F.cosine_similarity(ha, hn).item()*100
            sim_p.append(sp); sim_n.append(sn)
            if sp > sn: aciertos += 1

    print(f"  - Precisión Topológica: {(aciertos/pruebas)*100:.2f}%")
    print(f"  - Similitud Plagios: {sum(sim_p)/pruebas:.2f}%")
    print(f"  - Similitud Randoms: {sum(sim_n)/pruebas:.2f}%")
    return historial

# EJECUCIÓN
# ================================
if len(dataset_train) > 0:
    torch.manual_seed(42)
```

```python
    modelo_cnn = CNNClasicaFingerprint().to(device)
    hist_cnn = entrenar_y_evaluar(modelo_cnn, "CNN Clásica (MLP)", lr=0.001)

    torch.manual_seed(42)
    modelo_kan = CNN_KAN_Fingerprint().to(device)
    hist_kan = entrenar_y_evaluar(modelo_kan, "CNN + KAN", lr=0.0005)

    torch.manual_seed(42)
    modelo_alex = AlexNetFingerprint().to(device)
    hist_alex = entrenar_y_evaluar(modelo_alex, "AlexNet", lr=0.0005)

    torch.manual_seed(42)
    modelo_eff = EfficientNetFingerprint().to(device)
    hist_eff = entrenar_y_evaluar(modelo_eff, "EfficientNet-B0", lr=0.0001, weight_decay=1e-4)

    fig, ejes = plt.subplots(1, 4, figsize=(24, 5))
    graficas = [("CNN Clásica", hist_cnn), ("CNN + KAN", hist_kan), ("AlexNet", hist_alex), ("EfficientNet", hist_eff)]

    for idx, (nombre, hist) in enumerate(graficas):
        ejes[idx].plot(hist['train'], label='Train', color='blue', linewidth=2)
        ejes[idx].plot(hist['val'], label='Val', color='red', linestyle='--', linewidth=2)
        ejes[idx].set_title(nombre, fontweight='bold')
        ejes[idx].legend()
        ejes[idx].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

Listing 8: Adaptación topológica y entrenamiento sobre el espacio latente

```
Output:

 RESULTADOS FINALES (CNN Clásica (MLP))
- Precisión Topológica: 80.00%
- Similitud Plagios: 87.72%
- Similitud Randoms: -12.97%


 RESULTADOS FINALES (CNN + KAN)
- Precisión Topológica: 60.00%
- Similitud Plagios: 77.48%
- Similitud Randoms: 44.80%


 RESULTADOS FINALES (AlexNet)
- Precisión Topológica: 80.00%
- Similitud Plagios: 76.76%
- Similitud Randoms: 10.68%


 RESULTADOS FINALES (EfficientNet-B0)
- Precisión Topológica: 65.71%
- Similitud Plagios: 47.02%
- Similitud Randoms: -21.74%
```

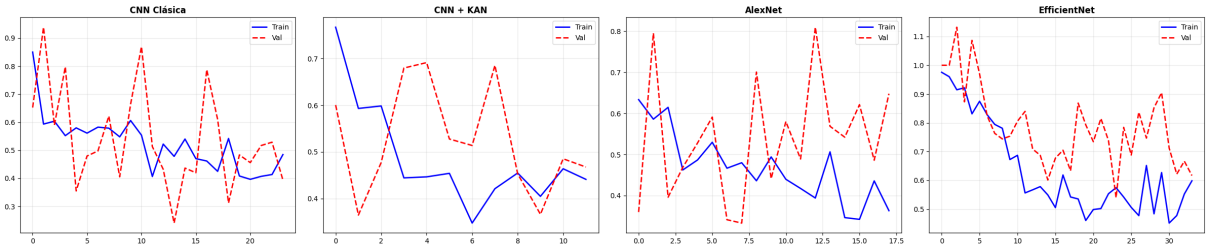


Figura 2: Curvas de aprendizaje para las cuatro arquitecturas evaluadas, mostrando la convergencia de la función *triplet margin loss*

5. Anatomía del *neural fingerprint*: el código de barras acústico

Para comprender el comportamiento del espacio latente optimizado por la red neuronal, resulta fundamental visualizar la semántica del *fingerprint* (el vector continuo $v \in \mathbb{R}^{128}$) generado por la capa de proyección final.

Al proyectar las componentes reales de estos vectores como mapas de calor bidimensionales (*heatmaps*), se hace observable el efecto geométrico impuesto por la *triplet margin loss* durante el entrenamiento. A continuación, se extrae del conjunto de *test* (datos no vistos previamente por el modelo) un caso de prueba compuesto por: una matriz original (*anchor*), su correspondiente versión interpretada (*positive*) y una pista acústicamente disonante seleccionada al azar (*negative*).

Si la optimización métrica ha convergido adecuadamente, los "códigos de barras" generados para la canción original y su *cover* exhibirán un patrón de activación análogo, mientras que la pista aleatoria mostrará un patrón divergente. Cabe destacar que, dado que los vectores latentes han sido forzados a residir en una hiperesfera de radio unitario mediante normalización L_2 en la última capa de la red, el cálculo de la similitud del coseno se simplifica computacionalmente a un producto escalar directo (producto punto) entre ambos vectores.

Implementación Computacional (Fase Final):
El siguiente bloque de código evalúa el modelo *EfficientNet-B0* entrenado, extrae los vectores de características en la memoria *CPU* y renderiza la comparativa visual de las firmas topológicas.

```
import seaborn as sns
import torch
import numpy as np
import random
import matplotlib.pyplot as plt

# Se evalua el modelo EfficientNet (mejor rendimiento empirico)
modelo_eff.eval()

with torch.no_grad():
    # 1. Extraccion de un trio aleatorio del conjunto de test
    idx_random = random.randint(0, len(dataset_test) - 1)
    anchor_img, positive_img, negative_img = dataset_test[idx_random]

    # 2. Generacion de los Fingerprints (vectores 128D) mediante inferencia
    fp_anchor = modelo_eff(anchor_img.unsqueeze(0).to(device)).cpu().numpy()
    fp_positive = modelo_eff(positive_img.unsqueeze(0).to(device)).cpu().numpy()
    fp_negative = modelo_eff(negative_img.unsqueeze(0).to(device)).cpu().numpy()

# 3. Calculo de la similitud del coseno real (Producto escalar sobre vectores L2
    )
similitud_plagio = np.dot(fp_anchor.flatten(), fp_positive.flatten()) * 100
similitud_random = np.dot(fp_anchor.flatten(), fp_negative.flatten()) * 100

# 4. Renderizado del grafico de calor
fig, ejes = plt.subplots(3, 1, figsize=(16, 6), gridspec_kw={'hspace': 0.8})
cmap = 'RdBu'
rango = 0.25

# Panel 1: Original
sns.heatmap(fp_anchor, ax=ejes[0], cmap=cmap, cbar=False, vmin=-rango, vmax=
    rango, xticklabels=False, yticklabels=False)
ejes[0].set_title("1. FINGERPRINT ORIGINAL (Anchor)", fontsize=12, fontweight='
```

```
    bold', color='black')

# Panel 2: Cover / Plagio
sns.heatmap(fp_positive, ax=ejes[1], cmap=cmap, cbar=False, vmin=-rango, vmax=
    rango, xticklabels=False, yticklabels=False)
ejes[1].set_title(f"2. FINGERPRINT COVER / PLAGIO (Similitud Reconocida: {
    similitud_plagio:.2f}%)", fontsize=12, fontweight='bold', color='darkblue')

# Panel 3: Aleatorio
sns.heatmap(fp_negative, ax=ejes[2], cmap=cmap, cbar=False, vmin=-rango, vmax=
    rango, xticklabels=False, yticklabels=False)
ejes[2].set_title(f"3. FINGERPRINT CANCION RANDOM (Similitud Reconocida: {
    similitud_random:.2f}%)", fontsize=12, fontweight='bold', color='darkred')

plt.suptitle("COMPARATIVA VISUAL DE VECTORES LATENTES EN $\\mathbb{R}^{128}$",
    fontsize=16, fontweight='black', y=1.05)
plt.show()
```

Listing 9: Visualización espectral cruzada de los vectores latentes (*heatmaps*)

Output:

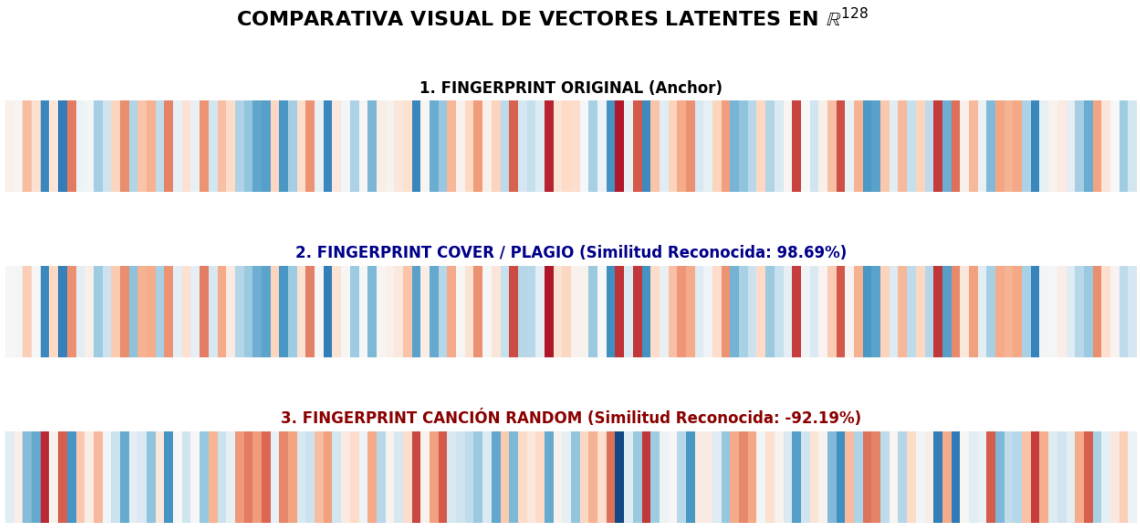


Figura 3: Comparativa visual de los vectores latentes (*heatmaps*) generados por la red. Las zonas de alta y baja activación (rojo y azul) funcionan como un código de barras estructural, permitiendo alinear semánticamente la versión original y su plagio, mientras que el ruido aleatorio mantiene ortogonalidad geométrica.

6. Implementación y Despliegue como Aplicación Web

Con el objetivo de hacer el sistema accesible y evaluable en condiciones reales, se ha desarrollado e implementado una aplicación web que integra el pipeline completo de inferencia: generación del espectrograma, extracción del *fingerprint* y búsqueda por similitud coseno contra la base de datos de embeddings precalculados:

- Arquitectura del sistema: El backend está implementado en Python con el framework FastAPI y sirve directamente la interfaz web. Al arrancar, carga en memoria el modelo entrenado (EfficientNet-B0) y el fichero `embeddings.pkl`, que contiene el vector de 128 dimensiones de cada canción del dataset. El sistema se despliega en contenedor Docker sobre Hugging Face Spaces, plataforma de computación en la nube que garantiza disponibilidad permanente y acceso público sin coste.
- Funcionamiento: El usuario accede a la aplicación desde cualquier navegador, incluyendo Chrome en dispositivos Android. Puede subir un archivo de audio (MP3, WAV, OGG, M4A o FLAC) o grabar directamente desde el micrófono del dispositivo. El servidor aplica exactamente el mismo pipeline de preprocesado utilizado durante el entrenamiento: cálculo de la STFT, conversión a escala logarítmica en decibelios, generación de la imagen PNG y normalización del tensor de entrada a 8 canales. El embedding resultante se compara por similitud coseno contra todos los embeddings del dataset, devolviendo las 5 canciones más similares ordenadas por porcentaje de coincidencia.
- Resultados sobre el dataset de entrenamiento: Las pruebas realizadas sobre canciones del propio dataset demuestran que el sistema identifica correctamente la canción original a partir de sus versiones y covers. La Figura 4 muestra un ejemplo representativo: por un lado, al introducir una canción original del dataset (izquierda), la aplicación la reconoce perfectamente; por otro lado al introducir un cover de una canción presente en el dataset (derecha), la aplicación sitúa la canción original en primera posición con una similitud superior al resto de resultados.

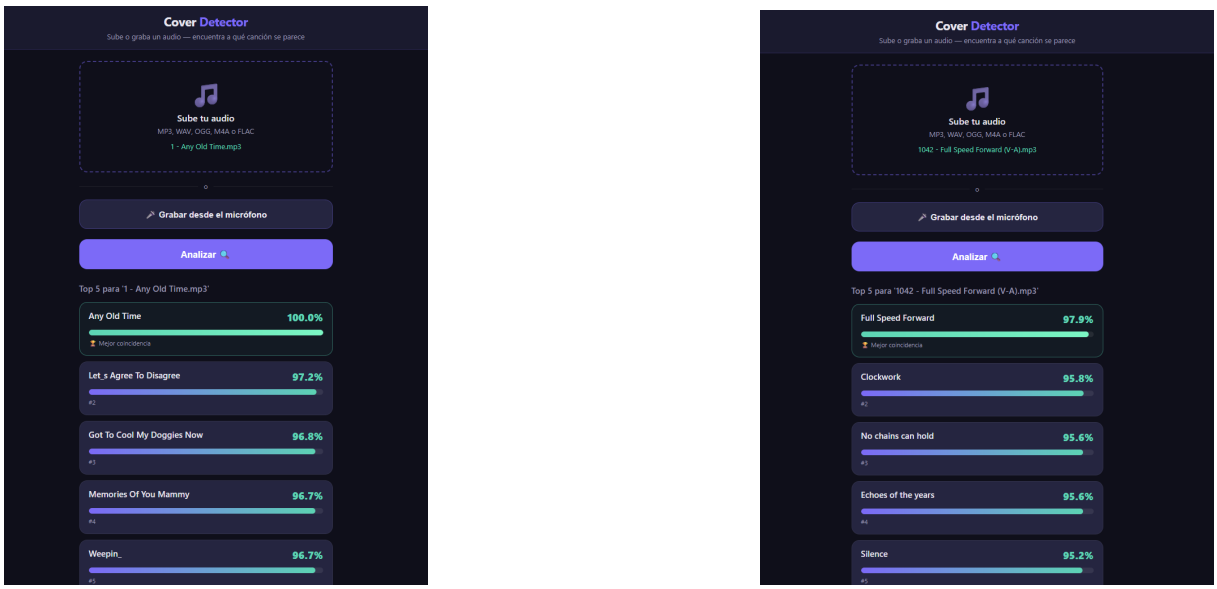


Figura 4: Pruebas de funcionamiento de la aplicación web